

Хранение строк и индексы

Table of contents

Хранение строк и индексы	1
Heap: строки лежат в отдельном файле	1
Clustered: таблица сама лежит в B-tree	2
Вторичные индексы всё равно живут отдельно	2
Heap vs clustered: цена по операциям	3
Почему этого недостаточно для аналитики	3

Хранение строк и индексы

Мы уже разобрали деревья: **B-tree** и **LSM** показали, как на диске живёт путь **ключ → значение**. Но реальная таблица хранит не один ключ, а **полные строки** из многих колонок. Поэтому следующий вопрос такой: где физически лежит строка целиком и как индекс до неё добирается.

Эта лекция про **строковое хранение**: **heap**, **clustered** и отдельные индексы, которые указывают на строки, а не заменяют их.

Неар: строки лежат в отдельном файле

Самый простой способ: завести файл под таблицу и складывать строки туда в том порядке, в котором они пришли. Такой файл называют **heap**. Не в смысле кучи-структуры данных, а в смысле «нет общего порядка». Так работает **PostgreSQL** по умолчанию.

Файл разбит на страницы по 8 КВ. Внутри страницы лежат сами строки и каталог слотов:

```
| page header  
| slot[0] → offset A  
| slot[1] → offset B  
| ...  
| (свободное место)  
| ...  
| [tuple B: row bytes]  
| [tuple A: row bytes]
```

У строки есть физический адрес **ctid = (page_id, slot)** — где именно она лежит на диске сейчас.

Если **поверх heap-таблицы** построен индекс, то этот индекс обычно хранит не полную строку, а **ссылку на строку в heap**. Например, `PRIMARY KEY` или индекс по `email` в `PostgreSQL` — это отдельный `B-tree`, и в листьях у него лежат пары `(key, ctid)`.

Любой поиск по индексу выглядит так:

1. спустились по индексу и нашли `ctid`;
2. сходили в `heap` по этому адресу и прочитали полную строку.

Цена `heap`-схемы — лишний прыжок `индекс → heap` на чтении. Выигрыш — дешёвый `INSERT`: строку можно положить почти в любую страницу со свободным местом.

Clustered: таблица сама лежит в B-tree

Вторая схема: раз по первичному ключу всё равно нужен `B-tree`, можно хранить в листьях дерева сразу **полные строки**. Так устроены `MySQL InnoDB`, `SQLite`, `Oracle IOT`.

Такую таблицу называют **clustered index** или **index-organized table**. Таблица физически упорядочена по первичному ключу.

На чтении это удобно: поиск по `PK` делает один спуск по дереву и сразу получает строку. Отдельного `heap`-файла нет.

На записи дороже:

- `INSERT` должен найти точное место в отсортированном дереве;
- при переполнении лист надо расщепить;
- случайный `PK` вроде `UUIDv4` даёт более болезненные вставки, чем монотонный ключ.

Из-за этого `clustered`-хранение обычно любит растущие ключи лучше, чем случайные.

Вторичные индексы всё равно живут отдельно

Даже если таблица `clustered`, **вторичный индекс не становится таблицей**. Он по-прежнему отдельная структура доступа.

В `PostgreSQL` вторичный индекс хранит что-то вроде:

`search key → ctid`

В `InnoDB` физический адрес строки ненадёжен: строки могут переехать между листьями дерева. Поэтому вторичный индекс хранит не адрес, а `PK`:

`secondary key → primary key → row`

Отсюда двойной спуск: сначала по вторичному индексу, потом по кластерному.

То же относится и к `hash index`: он тоже хранится **отдельно от таблицы**. Разница только в способе поиска:

`hash (key) → bucket → row locator`

Hash index хорош для поиска по точному равенству, но он не хранит строки «внутри себя» и не умеет диапазоны. Поэтому его полезно воспринимать не как другой способ хранения таблицы, а как ещё один отдельный путь доступа к тем же строкам.

Практический вывод простой: **каждый индекс — это отдельная структура, которую надо поддерживать при записи.**

Heap vs clustered: цена по операциям

Операция	Heap (PostgreSQL)	Clustered (InnoDB)
Поиск по PK	индекс → heap	один спуск до листа
Поиск по вторичному индексу	индекс → heap	вторичный → PK → кластерный
INSERT	в любую страницу со свободным местом	в точное место в сортировке, возможно расщепление
Диапазон по PK	случайный I/O по heap	последовательный проход по листьям
Чувствительность к форме PK	низкая	высокая

Грубо: heap удобнее для записи, clustered — для чтения по первичному ключу и диапазонам.

Почему этого недостаточно для аналитики

И heap, и clustered хранят строку целиком: все поля пользователя, заказа или события лежат рядом. Это идеальная раскладка для запроса вида:

```
SELECT * FROM users WHERE id = 123;
```

Но у аналитических запросов другая форма: они часто читают **мало колонок из очень многих строк**. Там строковая раскладка начинает тянуть с диска лишние данные.

Дальше отделим физическую организацию таблицы от типа индекса: B-tree, hash и bitmap отвечают на разные формы запросов, но каждый из них всё равно остаётся отдельным путём доступа к строкам.